ChatGPT

Разработка ПО с нуля через вайбкодинг

Executive summary

Этот гайд исходит из практической, а не романтизированной трактовки вайбкодинга: **не как “код сам собой появился из чата”**, а как **управляемая AI-assisted разработка**, где часть инженерной работы переносится с ручного написания кода на подготовку контекста, контрактов, планов, тестов и эксплуатационных контуров. В узком исходном смысле термин вырос из описанного Андреем Карпати режима “говорю модели, принимаю диффы, мало читаю код”; в более зрелой трактовке, которую поддерживают современные платформы, рабочим становится другой режим: *plan first, persistent knowledge, structured outputs, evals, review, rollback, observability*. Именно этот зрелый режим и лежит в основе предлагаемой методики. [1]

Главный вывод исследования такой: **вайбкодинг хорошо работает для старта, но плохо масштабируется без инженерных ограничителей**. Официальные гайды Lovable, Replit, Cursor, Codex, OpenAI, Anthropic и Google почти синхронно рекомендуют сначала формализовать задачу и ограничения, затем хранить устойчивые инструкции в knowledge/rules/AGENTS, использовать схемы и structured outputs, дробить работу на управляемые шаги, валидировать результат через тесты и evals и выводить агентные workflows в наблюдаемую среду с guardrails и rollback. [2]

Для production-ориентированной разработки особенно важны три вещи. Первая — **контекст-инжиниринг**, а не только prompt engineering: нужно проектировать, какие документы, схемы, память, tool definitions и traces доступны модели в каждый конкретный момент. Вторая — **формализованные контракты**: JSON Schema, tool/function calling, API schema, state model, error model и acceptance criteria. Третья — **непрерывная проверка**: deterministic tests, eval suites, browser/e2e-проверки, telemetry, release gates и rollback. Именно эти практики уменьшают скрытые дефекты, хаотичные переделки и перерасход токенов лучше, чем бесконечное переписывание “ещё более умных” промптов. [3]

Важно и то, что ощущение ускорения не всегда совпадает с реальной производительностью. В рандомизированном исследовании METR опытные open-source разработчики в ранних AI-инструментах 2025 года в среднем выполняли задачи **на 19% медленнее**, а не быстрее; исследование по багам в LLM-generated code описывает устойчивую таксономию ошибок вроде misinterpretation, missing corner case, hallucinated object и incomplete generation; исследования по security показывают, что качество и безопасность AI-кода существенно зависят от prompting, ручного guidance и языка/стека. Поэтому жизнеспособная методика должна закладывать валидацию заранее, а не “после демо”. [4]

Ниже я сознательно помечаю все неуказанные параметры проекта как **«не указано»** и предлагаю нейтральные варианты, не пытаясь навязать стек или платформу. Это важно, потому что выбор платформы, модели, бюджета и состава команды влияет на форму процесса сильнее, чем часто кажется на старте. Рекомендованный ниже процесс рассчитан прежде всего на **greenfield-проект**, где цель — получить **MVP или первый production-capable релиз** с разумным балансом между скоростью, качеством и стоимостью. Подход особенно хорошо подходит для web-приложений, внутренних инструментов, AI-assisted workflow products и B2B-сервисов; для

high-compliance или safety-critical систем требования к hardening и approvals должны быть жёстче. [5]

Параметр проекта	Статус	Нейтральные варианты
Целевая платформа	не указано	веб-приложение, внутренний инструмент, бот, desktop utility
Стек	не указано	TypeScript full-stack, Python backend + web frontend, mixed stack
Бюджет	не указано	лимит на токены, usage-based, enterprise budget control
Модель/провайдер	не указано	OpenAI, Anthropic, Google Vertex AI, смешанная маршрутизация
Команда	не указано	solo founder, founder + engineer, small team
Требования compliance	не указано	базовый security baseline, PII-aware, regulated environment
SLA/SLO	не указано	MVP-grade, internal tool, public beta, production-grade

Ниже — не просто обзор, а **готовая рабочая методика**: с фазами, артефактами, шаблонами файлов, чек-листами, roadmap, anti-patterns, мерой production-readiness и копируемым markdown-шаблоном гайда. Структура фаз соответствует вашему запросу: **Discovery/Problem framing → Contracts & Architecture → Project memory & prompts → Execution planning → Controlled implementation loops → Hardening & evals → Release & operate**. Эта последовательность хорошо согласуется и с современными IDE/агентами, и с практиками GenAIOps/MLOps, где prompt management, evaluation, testing, monitoring и tracing считаются частью нормального жизненного цикла. [6]

Что такое вайбкодинг и где метод работает

В исходном, узком смысле **вайбкодинг** — это режим, в котором разработчик описывает намерение естественным языком, принимает или почти принимает generated changes и скорее управляет результатом через обратную связь, чем вручную конструирует систему построчно. Именно такой режим Карпати описывал как “видеть, говорить, запускать, копипастить ошибки обратно”; в практическом кейсе MenuGen он показал, что этим способом реально можно дойти до работающего продукта с деплоем, auth и платежами, но при этом на реальном пути вылезают проблемы с API drift, интеграциями, rate limits, env vars, hosting и платежной связкой. [7]

Важно, однако, не размывать термин слишком сильно. Саймон Уиллисон специально возражал против употребления *vibe coding* как зонтика для любой AI-assisted разработки: если человек **читает, понимает, тестирует и может объяснить** результат, это уже не “чистый вайбкодинг”, а нормальная инженерная работа с LLM как ускорителем. Это различие полезно не ради терминологической чистоты, а потому, что оно определяет, **какие риски вы берёте на себя**: чистый вайбкодинг может быть приемлем для прототипа, но почти всегда опасен как модель сопровождения production-системы. [8]

Официальные платформы фактически подтверждают эту границу. Replit описывает vibe coding как практический, итеративный способ вести разработку через ясное руководство агентом; Lovable советует *plan before you prompt*, задавать уточняющие вопросы и строить по компонентам; Cursor и Codex делают акцент на plans, rules/AGENTS, review, skills и validation как штатной части агентной разработки. То есть зрелая индустриальная практика уже движется **от “accept all” к “steer, constrain, validate, observe”**. [9]

С практической точки зрения полезно различать три режима работы.

Подход	Как работает	Где уместен	Главный риск
Строгий вайбкодинг	естественный язык, быстрые прогоны, слабый контроль кода	идеи, weekend projects, throwaway prototypes	техдолг, скрытые дефекты, непредсказуемое сопровождение
Управляемый вайбкодинг	документы, планы, контракты, тесты, review	MVP, internal tools, early product build	ложное чувство готовности к production
AI-native engineering	agents + contracts + evals + CI/CD + observability + guardrails	production systems, долгоживущие продукты	избыточная сложность процесса без правильных метрик

Эта классификация — синтез полевых кейсов и официальных практик, а не формальный стандарт. Но именно она наиболее полезна для автора продукта: если ваш проект должен жить дольше первого демо, правильная цель — **не оставаться в strict-vibe режиме**, а как можно раньше перейти в управляемый режим и затем — при необходимости — в AI-native engineering. На это указывают и кейсы платформ, и исследования качества/безопасности, и наблюдения о производительности опытных разработчиков. [10]

Русскоязычная профессиональная среда пришла примерно к тем же выводам. На Хабре и в русскоязычных разборках вокруг термина повторяются две мысли: во-первых, вайбкодинг не отменяет инженерную дисциплину; во-вторых, инструмент полезен тем, кто умеет задавать ограничения и проверять результат, а не только тем, кто “умеет красиво просить”. Это не первоисточники уровня platform docs, но как региональный контекст они хорошо отражают реальное восприятие подхода русскоязычными разработчиками. [11]

Фазовая методика

Ниже предложена **семифазная методика**, которая превращает вайбкодинг из “чата ради скорости” в **документно-управляемый процесс разработки ПО с нуля**. Эта последовательность напрямую согласуется с платформенными советами: сначала определить цель и ограничения, затем зафиксировать архитектуру и контракты, затем подготовить устойчивую память проекта и prompts, после этого строить пошаговый план, исполнять его малыми циклами, прогонять hardening/evals и только потом выходить в release/operate. [12]

На диаграмме ниже — рекомендуемый контур процесса, который объединяет product framing, contracts, AI execution loop и эксплуатационный контур.

```
flowchart LR
    A[Проблема и ценность] --> B[User flows и acceptance criteria]
    B --> C[Контракты и архитектура]
    C --> D[Project memory и prompt registry]
    D --> E[Пошаговый план исполнения]
    E --> F[Кодогенерация малыми шагами]
    F --> G[Тесты, evals, self-review]
    G --> H[CI/CD и security gates]
    H --> I[Deploy, logs, traces, metrics]
    I --> J[Observe and refine]
    J --> B
```

Временные оценки ниже — **ориентир автора**, а не отраслевой стандарт. Они рассчитаны на greenfield-проект уровня MVP: одна основная пользовательская ценность, один основной datastore, до трёх внешних интеграций, без тяжёлого compliance. Если у вас mobile, payments, PII, медицинские/финансовые данные или высокий регуляторный контур, оценки для фаз hardening и release нужно увеличивать. То, что сами платформы и облачные архитектуры советуют считать testing, monitoring, tracing and governance частью нормального цикла, подтверждает, что такие запасы времени — не роскошь, а страховка. [13]

Discovery и problem framing

Эта фаза нужна, чтобы LLM не компенсировала продуктовую неопределённость красивым, но неверным решением. Официальные практики Lovable и Replit рекомендуют сначала определить, что получает пользователь, какие есть ограничения, и разрешить системе задать уточняющие вопросы перед тем, как писать код. Anthropic и OpenAI, в свою очередь, рекомендуют задавать success criteria и eval targets как можно раньше, иначе команда позже спорит уже не о реализации, а о том, что вообще считать “правильным результатом”. [14]

Поле	Содержание
Цель	зафиксировать проблему, пользователя, ценность, scope/out-of-scope, ручные и AI-assisted сценарии
Входы	идея продукта, исходные заметки, рыночный или внутренний контекст
Выходы и файлы	`docs/PRODUCT_BRIEF.md`, `docs/USER_FLOWS.md`, `docs/NFRS.md`, при необходимости `docs/ASSUMPTIONS.md`
Типовые роли	product owner, founder, UX; фактические исполнители: **не указано**
Критерий перехода	есть 3–5 ключевых user flows, acceptance criteria, ограничения первой версии и отдельное описание «что делает пользователь вручную» / «где помогает ИИ»
Оценка для MVP/solo	0.5–2 дня
Оценка для small team	1–3 дня

Главные риски этой фазы — утонуть в “генеральной стратегии” без артефактов, перепутать желаемую фичу с реальной пользовательской ценностью и позволить LLM самой домыслить domain rules. Лучший mitigation — держать продуктовые документы краткими и проверяемыми: user outcome, actors, flows, constraints, out-of-scope, NFRs. Это особенно важно, потому что исследования по багам в LLM-code показывают, что misinterpretation и missing corner cases — не редкие исключения, а типовые режимы отказа. [15]

Contracts и architecture

Эта фаза нужна, чтобы модель **не придумывала архитектуру заново на каждом ходу**. Современные инструменты сходятся в том, что устойчивый результат появляется, когда у агента есть заранее описанные границы: layout repo, coding standards, canonical examples, state model, contracts и команды проверки. Structured outputs и function/tool calling у OpenAI, strict tools и JSON outputs у Claude, а также function calling в Vertex AI показывают один и тот же инженерный паттерн: всё, что можно выразить через схему, нужно выражать через схему. [16]

Поле	Содержание
Цель	определить архитектуру модулей, контракты данных и поведения, state model, extension points
Входы	`PRODUCT_BRIEF`, `USER_FLOWS`, ограничения по платформе и данным
Выходы и файлы	`docs/ARCHITECTURE.md`, `docs/STATE_MODEL.md`, `docs/DOMAIN_MODEL.md`, `contracts/API_SCHEMA.yaml`, `contracts/TOOL_SCHEMAS.json`, `docs/DECISIONS.md`
Типовые роли	architect, tech lead, backend/frontend lead, AI engineer; фактические исполнители: **не указано**
Критерий перехода	основные интерфейсы формализованы; для критичных обменов есть явные схемы и error model; расширение функциональности возможно без переписывания ядра
Оценка для MVP/solo	1–3 дня
Оценка для small team	2–5 дней

Главные риски — копировать случайную архитектуру из output модели, смешивать бизнес-правила с UI-деталями и оставлять “серые зоны”, где API или tool вызовы не описаны. Mitigation: state model, interface matrix, canonical examples и ADR-lite в `DECISIONS.md`. Это уменьшает и баги, и стоимость последующих итераций, потому что schema violations и неверные tool arguments перестают лечиться через дорогие retries. [17]

Project memory и prompts

Эта фаза превращает разрозненные документы в **управляемую память проекта**. Anthropic прямо описывает этот уровень как context engineering: важен уже не только текст prompt-а, а весь набор данных, памяти, tool definitions и handoff artifacts, доступных модели. Lovable knowledge, AGENTS.md в Codex, rules/plans в Cursor и reusable workflows/skills у Replit/Codex делают один и тот

же акцент: устойчивые инструкции должны жить в проекте и версиях, а не только в истории чатов. [18]

Поле	Содержание
Цель	собрать устойчивый контекст: project rules, knowledge, prompt registry, индекс артефактов и правила их подключения
Входы	архитектурные документы, контракты, coding standards, команды build/test/lint
Выходы и файлы	`docs/INDEX.md`, `AGENTS.md`, `prompts/PROMPT_REGISTRY.md`, `docs/KNOWLEDGE.md`, при необходимости `prompts/*.md`
Типовые роли	tech lead, AI engineer, senior dev; фактические исполнители: **не указано**
Критерий перехода	есть краткие правила для агента, карта контекста и versioned prompts; понятно, какие документы подмешиваются в какие задачи
Оценка для MVP/solo	0.5–2 дня
Оценка для small team	1–3 дня

Главные риски — сделать “кладбище markdown-файлов”, которые никто не обновляет, и кормить модели всем репозиторием на каждом ходу. Mitigation: `INDEX.md` с полями *purpose / include-when / exclude-when / owner / update rule*, короткий `AGENTS.md`, versioned `PROMPT_REGISTRY.md`, а также cache-friendly организация постоянного префикса prompt-а. Exact prefix matters и для OpenAI prompt caching, и для Claude prompt caching, а Google рекомендует context caching ровно для случаев, где большой начальный контекст повторно используется во многих запросах. [19]

Execution planning

Эта фаза нужна, чтобы путь от документов к коду был **явным и inspectable**, а не оставался в голове автора или внутри одного длинного чата. Lovable Plan mode, Codex workflows, Cursor plan practices и Replit vibe tutorials все рекомендуют сначала сформировать formal plan, проверить его и только потом переключаться в режим изменений. Особенно важно, чтобы план был не просто списком желаний, а рабочим документом со статусами, milestone, dependencies, risks и evidence. [20]

Поле	Содержание
Цель	превратить видение и архитектуру в исполнимый поэтапный план
Входы	все документы предыдущих фаз
Выходы и файлы	`docs/EXEC_PLAN.md`, `docs/TASK_BOARD.md`, `docs/RISK_REGISTER.md`, при необходимости `docs/MILESTONES.md`
Типовые роли	tech lead, founder, PM; фактические исполнители: **не указано**
Критерий перехода	каждая крупная задача разбита на small task slices; у каждого шага есть definition of done, dependencies и rollback point

Поле	Содержание
Оценка для MVP/solo	0.5–1.5 дня
Оценка для small team	1–2 дня

Главные риски — слишком крупные задачи, отсутствие зависимостей и попытка реализовать несколько смен контекста одним прогоном. Mitigation: small task slices, milestone gates, отдельные задачи на business logic, infra, UI, contracts и testing. Cursor отдельно отмечает, что сегодняшние агенты лучше работают на **focused tasks**, а complex projects требуют координации и планирования. [21]

Controlled implementation loops

Это центральная фаза: код пишется **малыми подтверждаемыми циклами**, а не одной большой магической инструкцией. Лучшие практики Codex, Cursor, Replit и Lovable сходятся на похожем loop: пересказ задачи → уточняющие вопросы → план изменений → минимальная реализация → self-review → тесты/проверки → фиксация результата → rollback при необходимости. Replit отдельно рекомендует checkpoints and rollbacks; Cursor и Codex — review и validation; Lovable — Plan mode для сложных изменений и debugging; Replit Agent 3 добавляет browser-based self-testing и code optimization/self-review. [22]

Поле	Содержание
Цель	реализовывать фичи малыми циклами с верификацией на каждом шаге
Входы	активный task slice, часть контекста из `INDEX.md`, relevant contracts, active prompt version
Выходы и файлы	код, tests, migrations, PR/commit, `docs/CHANGE_LOG_DEV.md`, обновления `EXEC_PLAN.md`
Типовые роли	developer, AI engineer, reviewer/QA; фактические исполнители: **не указано**
Критерий перехода	каждый шаг проверен build/test/lint/schema checks; есть traceable evidence и понятный next step
Оценка для MVP/solo	2–10 дней
Оценка для small team	3–15 дней

Главные риски — “accept all” на больших diff-ах, смешение новой логики и рефакторинга, попытка исправить сломанную ветку бесконечными чат-командами вместо rollback. Исследование METR хорошо объясняет, почему такие циклы у опытных разработчиков могут замедлять работу: экономия времени на письме кода легко съедается временем на prompting, ожидание генерации, review, cleanup и rework. Поэтому controlled loops и rollback discipline критически важны. [23]

Hardening и evals

Эта фаза отделяет “уже работает” от “можно доверять”. OpenAI прямо рекомендует continuous evaluation и сравнение изменений на eval suites, а не “vibe-based evals”; Anthropic советует сначала определить success criteria, затем строить datasets и grader logic; Replit и Lovable добавляют browser/app testing и systematic debugging; Cursor подчёркивает correctness, code quality, efficiency и interaction behavior как разные измерения качества агентной системы. [24]

Поле	Содержание
Цель	довести систему до стабильного и измеримого качества
Входы	работающее приложение, tests baseline, traces, known issues
Выходы и файлы	`docs/TEST_CATALOG.md`, `evals/eval_dataset.jsonl`, `docs/QUALITY_SCORECARD.md`, `docs/KNOWN_LIMITATIONS.md`
Типовые роли	QA, AI engineer, developer, product owner; фактические исполнители: **не указано**
Критерий перехода	критические user flows проходят; regression suite зелёная; security defects не блокируют релиз
Оценка для MVP/solo	1–4 дня
Оценка для small team	2–7 дней

Главные риски — сводить hardening к ручному “покликал — вроде живо”, не иметь adversarial/edge cases и не фиксировать новые найденные случаи в eval dataset. Исследования по багам и security LLM-generated code показывают, что ошибки часто лежат именно в углах: corner cases, wrong input type, hallucinated objects, unsafe defaults. Hardening без datasets и checks оставляет эти режимы почти неизбежными. [25]

Release и operate

Эта фаза делает систему **эксплуатационно пригодной**. GitHub Actions и GitLab CI/CD дают стандартную основу для build-test-deploy-monitor; OpenTelemetry объединяет traces, metrics и logs с общей корреляцией; OpenAI Agents SDK позволяет видеть model calls, tool calls, handoffs, guardrails и custom spans; Azure Well-Architected и Google GenAIOps/MLOps guidance подчёркивают, что monitoring, tracing, evaluation и governance должны жить рядом с релизным циклом, а не после него. [26]

Поле	Содержание
Цель	подготовить безопасный, наблюдаемый, откатываемый релиз и режим сопровождения
Входы	результаты hardening, CI baseline, release checklist, telemetry spec
Выходы и файлы	`.github/workflows/*` или `.gitlab-ci.yml`, `docs/RELEASE_CHECKLIST.md`, `docs/RUNBOOK.md`, `docs/OBSERVABILITY.md`, dashboards/alerts

Поле	Содержание
Типовые роли	DevOps/platform, reviewer, security, product; фактические исполнители: **не указано**
Критерий перехода	есть staged release, rollback, structured logs, traces, metrics, budget/usage controls, guardrails и approvals
Оценка для MVP/solo	1–3 дня
Оценка для small team	2–5 дней

Главные риски — выкатывать AI-heavy приложение без трассировки, approval gates, secret hygiene и rollback-плана. Для LLM-приложений это особенно опасно, потому что OWASP Top 10 for LLM Applications 2025 прямо относит prompt injection к числу ключевых рисков, а platform docs рекомендуют guardrails, human review, sandbox separation и least privilege как базовые меры защиты. [27]

Полезно закладывать roadmap в явном виде, а не только в голове автора. Ниже — ориентировочная timeline-диаграмма для solo/MVP режима.

```
timeline
    title Roadmap разработки ПО с нуля через управляемый вайбкодинг
    День 1 : PRODUCT_BRIEF
           : USER_FLOWS
           : NFRS
    День 2 : ARCHITECTURE
           : STATE_MODEL
           : DOMAIN_MODEL
    День 3 : API_SCHEMA
           : TOOL_SCHEMAS
           : SECURITY_BASELINE
           : AGENTS.md
    День 4 : INDEX.md
           : PROMPT_REGISTRY
           : EXEC_PLAN
           : TEST_CATALOG
    День 5 : CI baseline
           : smoke tests
           : первый controlled loop
    Неделя 2 : integration
             : regression evals
             : observability
             : release hardening
    Неделя 3 : staged rollout
             : metrics review
             : bug triage
             : refine
```

Артефакты, структура репозитория и рабочие шаблоны

Если коротко, то лучший способ уменьшить хаос и расход токенов — не “писать ещё более длинные промпты”, а держать у проекта **небольшой, хорошо индексируемый пакет артефактов**. Именно в эту сторону смотрят current platform patterns: workspace/project knowledge в Lovable, AGENTS.md и skills в Codex, rules/plans в Cursor, explicit project structure и version control в Replit. Все они по сути решают одну задачу — сделать проектный контекст **устойчивым, повторно используемым и reviewable**. [28]

Рекомендуемая структура greenfield-репозитория может выглядеть так:

```
repo/
  AGENTS.md
  README.md
  .gitignore
  .github/
    workflows/
      ci.yml
      deploy.yml
  app/
  web/
  services/
  packages/
  contracts/
    API_SCHEMA.yaml
    TOOL_SCHEMAS.json
    EVENTS_SCHEMA.yaml
  docs/
    INDEX.md
    PRODUCT_BRIEF.md
    USER_FLOWS.md
    NFRS.md
    ARCHITECTURE.md
    STATE_MODEL.md
    DOMAIN_MODEL.md
    DECISIONS.md
    EXEC_PLAN.md
    TASK_BOARD.md
    RISK_REGISTER.md
    TEST_CATALOG.md
    SECURITY_BASELINE.md
    OBSERVABILITY.md
    RELEASE_CHECKLIST.md
    RUNBOOK.md
    CHANGE_LOG_DEV.md
    KNOWN_LIMITATIONS.md
  prompts/
    PROMPT_REGISTRY.md
    system/
    tasks/
```

```
    debugging/
  evals/
    eval_dataset.jsonl
    graders/
  tests/
    unit/
    integration/
    e2e/
    fixtures/
  scripts/
```

Для управления ролями и ownership удобно использовать простую матрицу артефактов. Фактические исполнители у вас **не указаны**, поэтому ниже — типовые роли.

Артефакт	Назначение	Типовая роль
`PRODUCT_BRIEF.md`	цель продукта, scope, пользователи, ценность	product owner / founder
`USER_FLOWS.md`	happy path, edge/failure cases, ручной и AI-assisted flow	product / UX
`NFRS.md`	latency, privacy, cost, reliability, logging	architect / founder
`ARCHITECTURE.md`	модули, data flow, extension points	architect / tech lead
`STATE_MODEL.md`	UI/session/job/tool states и transitions	frontend/backend
`DOMAIN_MODEL.md`	сущности, инварианты, lifecycle	backend
`API_SCHEMA.yaml`	API contracts	backend
`TOOL_SCHEMAS.json`	tool/function contracts для LLM	AI engineer / backend
`INDEX.md`	карта контекста: что, когда и зачем подмешивать	tech lead / AI engineer
`AGENTS.md`	устойчивые правила поведения агента	tech lead
`PROMPT_REGISTRY.md`	versioned prompts и правила использования	AI engineer
`EXEC_PLAN.md`	пошаговый план, milestone и status	tech lead / founder
`TEST_CATALOG.md`	catalog тестов и evals	QA / AI engineer
`SECURITY_BASELINE.md`	secrets, approvals, allowed tools/data	security / devops
`OBSERVABILITY.md`	logs, traces, metrics, dashboards	platform / devops
`RELEASE_CHECKLIST.md`	release gate и rollback	devops / release owner

Ниже — **расширенные шаблоны** для ключевых файлов. Это не “официальный стандарт”, а практический набор, собранный из AGENTS/knowledge/rules/evals/guardrails практик разных

платформ. Они намеренно сделаны короткими, чтобы быть и полезными человеку, и удобными для LLM в качестве контекста. [29]

Шаблон INDEX.md

```
# INDEX

## Назначение
Карта проектного контекста: какие документы существуют, когда их включать в
контекст модели и кто отвечает за обновление.

## Правила использования
- Не включать все документы в каждый запрос.
- Сначала подключать AGENTS.md + relevant docs.
- Для больших задач сначала подключать PRODUCT_BRIEF, ARCHITECTURE,
EXEC_PLAN.
- Для багов подключать CHANGE_LOG_DEV, TEST_CATALOG, relevant logs/traces.
- Для API/tool-задач подключать contracts/*.

## Карта артефактов

| Файл | Purpose | Include when | Exclude when | Owner | Update rule |
|---|---|---|---|---|---|
| docs/PRODUCT_BRIEF.md | Цель и scope продукта | Новая фича, product
decision | Мелкий refactor | product | при изменении ценности/границ |
| docs/ARCHITECTURE.md | Модули и связи | Архитектурные задачи | Copy updates
| tech lead | при изменении модулей |
| contracts/API_SCHEMA.yaml | API contracts | API/backend work | UI copy |
backend | при изменении контрактов |
| prompts/PROMPT_REGISTRY.md | Prompt versions | AI workflow changes | infra-
only задачи | AI engineer | при изменении prompt logic |

## Минимальные контекст-пакеты
### Новый feature slice
- AGENTS.md
- PRODUCT_BRIEF.md
- USER_FLOWS.md
- relevant ARCHITECTURE section
- EXEC_PLAN.md
- relevant contracts

### Диагностика бага
- AGENTS.md
- CHANGE_LOG_DEV.md
- TEST_CATALOG.md
- relevant logs/traces
- relevant files diff summary
```

Шаблон AGENTS.md

```
# AGENTS

## Миссия
Работай как инженер в существующем проекте. Сохраняй архитектурные границы,
не расширяй scope без явного запроса, предпочитай минимальные изменения.

## Репозиторий
- Основные директории: <описать>
- Канонические примеры: <пути>
- Ключевые зависимости: <список>

## Команды
- Build: <command>
- Test: <command>
- Lint: <command>
- Typecheck: <command>
- E2E: <command>

## Правила изменений
- Сначала кратко перескажи задачу.
- Если есть критичная неопределённость, задай вопросы до изменения кода.
- Сначала покажи план по файлам.
- Меняй минимально необходимое количество файлов.
- Не смешивай новую фичу и крупный рефакторинг в одном шаге.
- Если меняется контракт — обнови schemas и тесты.
- Если меняется архитектура — обнови DECISIONS.md.

## Запреты
- Не добавляй зависимости без явной необходимости.
- Не меняй секреты, окружение и CI без отдельного указания.
- Не удаляй тесты, если нет явного запроса.
- Не выполняй рискованные действия без approval.

## Формат ответа после изменений
- Что изменено
- Какие файлы затронуты
- Какие проверки выполнены
- Какие риски остались
- Что проверить вручную

## Definition of Done
- Код собирается
- Тесты проходят
- Контракты не нарушены
- Риски зафиксированы
- Документация обновлена при необходимости
```

Шаблон PROMPT_REGISTRY.md

```
# PROMPT REGISTRY

## Правила
- Каждый prompt имеет id и версию.
- Изменение prompt logic требует changelog entry.
- Для каждого prompt указать eval cases и expected failure modes.
- Стабильный префикс должен меняться как можно реже.

## Реестр

### prompt_id: feature_plan_v1
- purpose: спланировать реализацию фичи до написания кода
- owner: не указано
- model: не указано
- inputs:
  - PRODUCT_BRIEF
  - USER_FLOWS
  - ARCHITECTURE excerpt
  - current EXEC_PLAN
- output contract:
  - summary
  - questions
  - file plan
  - risks
  - done criteria
- include_when:
  - новая фича
  - архитектурное изменение
- eval cases:
  - корректно выявляет неопределённости
  - не предлагает менять лишние файлы
- last_changed: YYYY-MM-DD

### prompt_id: bug_diagnosis_v1
- purpose: сначала диагностировать, потом исправлять
- owner: не указано
- inputs:
  - error logs
  - CHANGE_LOG_DEV
  - TEST_CATALOG
- output contract:
  - top causes
  - validation steps
  - minimal fix plan
  - verification plan
```

Шаблон PRODUCT_BRIEF.md

```
# PRODUCT BRIEF

## Параметры
- Платформа: не указано
- Стек: не указано
- Бюджет: не указано
- Команда: не указано
- Compliance: не указано

## Проблема
<какую проблему решаем>

## Пользователь
<кто пользователь и в каком контексте использует продукт>

## Ценность
<какой измеримый результат получает пользователь>

## Что пользователь делает вручную
<manual flow>

## Где помогает ИИ
<AI-assisted flow>

## Первая версия
### In scope
- <...>

### Out of scope
- <...>

## Критические сценарии
1. <happy path>
2. <edge case>
3. <failure case>

## Acceptance criteria
- <...>
- <...>

## Нефункциональные ограничения
- latency:
- privacy:
- reliability:
- budget:
```

Шаблон EXEC_PLAN.md

```
# EXECUTION PLAN

## Цель релиза
<что должно быть готово и как понять, что это работает>

## Milestones
| Milestone | Scope | Status | Evidence | Risk |
|---|---|---|---|---|
| M1 | Foundation | todo |  |  |
| M2 | Core workflow | todo |  |  |
| M3 | Hardening | todo |  |  |
| M4 | Release | todo |  |  |

## Task slices
### T-001
- goal:
- files:
- dependencies:
- contract impact:
- test impact:
- done when:
- rollback point:

### T-002
- goal:
- files:
- dependencies:
- contract impact:
- test impact:
- done when:
- rollback point:

## Open questions
- <...>

## Known risks
- <...>

## Changes log
- YYYY-MM-DD: <что изменилось в плане и почему>
```

Шаблон TEST_CATALOG.md

```
# TEST CATALOG

## Правила
- У каждой фичи минимум один happy path, один edge case, один failure case,
```

```
один regression case.
- Если меняется контракт — обновить schema tests.
- Если меняется AI behavior — обновить eval dataset.

## Матрица тестов
| ID | Type | Scenario | Input/Fixture | Oracle | Owner | Status |
|---|---|---|---|---|---|---|
| U-001 | unit | ... | ... | ... | ... | todo |
| I-001 | integration | ... | ... | ... | ... | todo |
| E-001 | e2e | ... | ... | ... | ... | todo |
| A-001 | ai-eval | ... | ... | ... | ... | todo |

## Adversarial / edge cases
- пустой ввод
- неверный тип данных
- повторный запрос / retry
- конфликтующее состояние
- попытка prompt injection
- превышение лимитов
```

Шаблон SECURITY_BASELINE.md

```
# SECURITY BASELINE

## Данные
- Какие данные обрабатываются:
- Какие данные нельзя отправлять модели:
- Какие данные требуют маскирования:

## Секреты
- Где хранятся секреты:
- Кто имеет доступ:
- Как ротируются:

## Инструменты и действия
- Какие tool calls разрешены:
- Какие действия требуют approval:
- Какие действия запрещены:

## Среды
- dev:
- staging:
- prod:

## Минимум проверок
- dependency scan
- secret scan
- auth/access review
- basic OWASP LLM checks
- prompt injection review
```

```
- logging/privacy review

## Инциденты
- как обнаружить
- как ограничить ущерб
- как откатить
```

Шаблон DoR и DoD

Definition of Ready и Definition of Done должны быть короткими, но обязательными. Иначе они перестают работать как gate и превращаются в неисполняемую “мудрость в документе”. Практика definition-driven workflows напрямую поддерживается рекомендациями Codex про explicit definition of done, eval-driven guidance OpenAI/Anthropic и release-oriented подходами DevOps/GenAIOps. [30]

```
# Definition of Ready
- [ ] PRODUCT_BRIEF актуален
- [ ] USER_FLOWS описаны
- [ ] ARCHITECTURE и contracts существуют
- [ ] AGENTS.md создан
- [ ] EXEC_PLAN содержит task slice
- [ ] TEST_CATALOG содержит минимум happy/edge/failure/regression
- [ ] mock/fixture данные подготовлены
- [ ] security ограничения понятны
- [ ] команда build/test/lint/typecheck известна

# Definition of Done
- [ ] задача выполнена в границах согласованного task slice
- [ ] build проходит
- [ ] tests проходят
- [ ] schema/contracts валидны
- [ ] известные риски зафиксированы
- [ ] EXEC_PLAN обновлён
- [ ] CHANGE_LOG_DEV обновлён
- [ ] manual checks выполнены
```

Практики качества, снижения токен-расхода и production-readiness

Лучшие практики здесь работают слоями: **меньше неопределённости → меньше циклов переделки → меньше токенов → меньше багов**. Это хорошо видно и в официальных docs, и в полевых наблюдениях. OpenAI прямо советует continuous evals, model selection по балансу accuracy/latency/cost, prompt caching, compaction, tool search, batch/flex для офлайн-задач и observability в агентных SDK; Anthropic рекомендует context engineering, memory/compaction, structured outputs, programmatic tool calling и тестирование на success criteria; Google рекомендует context caching, prompt design и function calling через schema-совместимые описания. [31]

Практики снижения багов и стоимости

Техника	Как уменьшает баги	Как уменьшает токены/ стоимость	Где использовать
Prompt versioning	стабилизирует поведение и rollback prompt logic	убирает хаотические повторные объяснения	все AI-heavy проекты
Prompt caching	меньше дрейфа в стабильном префиксе	снижает стоимость и latency повторяющихся запросов	длинные постоянные префиксы
Context compaction	уменьшает context rot в долгих сессиях	снижает размер активного окна	длинные workflows
Model routing	не тратит сильную модель на простые подзадачи	снижает среднюю цену запроса	mixed workloads
Batch/Flex	позволяет гонять evals и офлайн jobs дешевле	50% lower cost у Batch, lower-cost async flows у Flex	массовые офлайн операции
Small task slices	легче локализовать дефект и откатить	меньше размер контекста и стоимость неудачного шага	всегда
Structured outputs	убирают format/schema failures	меньше retries и ручного парсинга	API/tool/JSON-tasks
Auto-tests	ловят регрессии до релиза	уменьшают дорогие ручные циклы	всегда
Dialog simulation	ловит UX/LLM regressions в conversation flow	удешевляет раннюю проверку сценариев	чатовые и agentic apps
Tool search/ deferred loading	уменьшает лишний tool overhead	меньше токенов на tool definitions	большие tool surfaces

Несколько практик особенно окупаются. **Prompt caching** у OpenAI и Claude требует стабильного повторяющегося префикса; у OpenAI cache hits завязаны на exact prefix matches, у Claude есть automatic и explicit caching, а Vertex AI рекомендует context caching именно для сценариев с большим общим начальным контекстом. **Compaction** в OpenAI и context/memory-техники Anthropic позволяют держать долгие workflows без экспоненциального роста стоимости и деградации качества. **Tool search** и programmatic tool calling уменьшают overhead от огромных tool definitions и intermediate tool results. [32]

Ниже — рекомендуемый operational loop качества.

```
flowchart LR
    A[Plan] --> B[Generate]
```

```
    B --> C[Validate]
    C --> D[Observe]
    D --> E[Refine]
    E --> A
    C --> F[Rollback]
    F --> A
```

Этот цикл нужен потому, что генеративные системы по определению вариативны. OpenAI отдельно подчёркивает, что для AI architectures традиционных deterministic tests недостаточно и нужны evals; Anthropic рекомендует строить success criteria и grader-логику, а не надеяться на ощущения; Cursor дополнительно измеряет correctness, code quality, efficiency и interaction behavior как разные оси качества. [33]

Production-readiness

Production-readiness для вайбкодинг-проекта означает не “LLM написал более-менее симпатичный код”, а то, что система **управляема, наблюдаема, откатываема и защищена**. В минимальный production baseline стоит включить следующее.

Область	Минимальное требование	Основание
CI/CD	автоматические build, test, deploy, monitor pipelines	GitHub Actions и GitLab CI/CD описывают это как базовый цикл поставки [34]
Observability	traces + metrics + logs, коррелированные по request/correlation IDs	OpenTelemetry даёт единый vendor-neutral каркас; OpenAI Agents SDK даёт traces model/tool/handoff workflow-ов [35]
Logging	structured logs с привязкой к версии релиза и user/session context	это снижает стоимость диагностики и делает LLM workflow inspectable [36]
Metrics	task success, regression pass rate, cost per successful task, retry rate, p95 latency	eval-driven и production-monitoring guidance у OpenAI/Azure Foundry/Anthropic [37]
Rollback	откат к checkpoint/release, миграции с безопасным rollback path	Replit checkpoints/rollbacks и обычные CI/CD практики [38]
Guardrails	input/output/tool checks, policy checks, approvals	OpenAI guardrails and human review guidance [39]
Approvals	человек подтверждает destructive или high-risk actions	соответствует human-in-the-loop practices и LLM security guidance [40]
Sandbox	изоляция исполнения от orchestration layer	OpenAI sandbox guidance и security patterns для agentic execution [41]
Security	prompt injection mitigation, secret hygiene, least privilege	OWASP Top 10 for LLM Applications 2025 [42]

Anti-patterns и mitigation

Самые вредные anti-patterns почти всегда повторяются из проекта в проект.

Anti-pattern	Почему вредно	Mitigation
“Сначала быстро нагенерим, потом разберёмся”	технический долг переносится в production	contracts-first, DoR/DoD, release gates
“Один огромный master prompt на всё”	высокий token burn, context drift, слабая локализация ошибок	context packages + INDEX.md + task slicing
“Тесты потом”	дефекты становятся системными и дорогими	TEST_CATALOG + baseline unit/integration/e2e/evals
“Все документы в каждый запрос”	лишняя стоимость и шум	selective inclusion, compaction, caching
“Сильная модель для каждой подзадачи”	завышенная стоимость и latency	model routing
“Исправим сломанную ветку ещё пятью чат-командами”	усиливает дрейф и сложность	rollback к checkpoint и пересмотр контракта/плана
“Никаких approvals, агент сам справится”	высокий риск destructive side effects	human review и guardrails
“Секреты и PII можно пока не трогать”	security debt и возможные утечки	SECURITY_BASELINE + secret scan + policy review

Эти anti-patterns особенно опасны потому, что research и практика показывают: LLM часто создаёт **правдоподобность без достаточной надёжности**. Отсюда и классические режимы отказа: баги в углах, небезопасные defaults, review overhead, медленные agentic loops и давление “давайте уже выкатывать, ведь почти работает”. Именно поэтому disciplined AI engineering — это не противоположность вайбкодингу, а его естественное взросление. [43]

Готовый markdown-файл гайда

Ниже — **копируемый markdown-файл**, который можно сохранить как `AI_NATIVE_VIBE_CODING_GUIDE.md`. Он не пытается заменить всю техническую документацию проекта; его задача — стать главным рабочим документом процесса, который связывает продуктовые артефакты, архитектуру, prompts, план, качество и релизные требования. Шаблон ниже — практический синтез подходов из official docs и кейсов, перечисленных в отчёте. [44]

```
# Разработка ПО с нуля через вайбкодинг

## Executive summary
Этот документ описывает, как вести разработку ПО с нуля через управляемый AI-assisted процесс так, чтобы:
- уменьшать количество багов;
```

- уменьшать объём переделок;
- снижать расход токенов;
- получать жизнеспособное, сопровождаемое приложение.

Под “вайбкодингом” в этом документе понимается не хаотическая генерация кода, а управляемая разработка, где:
- сначала формализуется продукт;
- затем задаются архитектура и контракты;
- затем создаётся память проекта и реестр промптов;
- затем реализация идёт малыми, проверяемыми циклами;
- затем выполняются hardening, evals и выпуск.

Параметры проекта
- Платформа: не указано
- Стек: не указано
- Бюджет: не указано
- Команда: не указано
- Compliance: не указано
- SLA/SLO: не указано

Основные принципы
- Сначала продукт, потом код.
- Сначала контракты, потом реализация.
- Один шаг = один измеримый результат.
- Устойчивые инструкции живут в файлах проекта, а не только в чате.
- В каждый запрос включается только нужный контекст.
- Любая задача заканчивается проверкой, а не “ощущением, что ок”.
- Любое опасное действие требует подтверждения.
- Любую неудачную итерацию легко откатить.

Структура репозитория
```text

repo/
  AGENTS.md
  .github/
    workflows/
      ci.yml
      deploy.yml
  contracts/
    API_SCHEMA.yaml
    TOOL_SCHEMAS.json
  docs/
    INDEX.md
    PRODUCT_BRIEF.md
    USER_FLOWS.md
    NFRS.md
    ARCHITECTURE.md
    STATE_MODEL.md
    DOMAIN_MODEL.md
    DECISIONS.md
```

```
    EXEC_PLAN.md
    TASK_BOARD.md
    RISK_REGISTER.md
    TEST_CATALOG.md
    SECURITY_BASELINE.md
    OBSERVABILITY.md
    RELEASE_CHECKLIST.md
    RUNBOOK.md
    CHANGE_LOG_DEV.md
    KNOWN_LIMITATIONS.md
  prompts/
    PROMPT_REGISTRY.md
  evals/
    eval_dataset.jsonl
  tests/
    unit/
    integration/
    e2e/
    fixtures/
```

Фаза Discovery и problem framing

Цель

Определить, какую проблему решает продукт, кто пользователь, какой результат он получает, что он делает вручную и где подключается ИИ.

Входы

- идея;
- заметки;
- исходные требования;
- пользовательский или бизнес-контекст.

Выходы

- docs/PRODUCT_BRIEF.md
- docs/USER_FLOWS.md
- docs/NFRS.md
- при необходимости docs/ASSUMPTIONS.md

Gate перехода

- описаны 3–5 основных сценариев;
- есть happy path, edge cases и failure cases;
- разделены manual flow и AI-assisted flow;
- зафиксирован out-of-scope.

Фаза Contracts & Architecture

Цель

Зафиксировать устройство системы до активной кодогенерации.

Входы

- PRODUCT_BRIEF;
- USER_FLOWS;
- NFRS.

Выходы

- docs/ARCHITECTURE.md
- docs/STATE_MODEL.md
- docs/DOMAIN_MODEL.md
- contracts/API_SCHEMA.yaml
- contracts/TOOL_SCHEMAS.json
- docs/DECISIONS.md

Gate перехода

- основные модули и связи описаны;
- критичные контракты формализованы;
- ошибки и состояния описаны;
- заложены точки расширения.

Фаза Project memory & prompts

Цель

Создать устойчивую память проекта и versioned prompts.

Входы

- архитектурные и продуктовые документы;
- coding standards;
- команды проверки.

Выходы

- docs/INDEX.md
- AGENTS.md
- prompts/PROMPT_REGISTRY.md
- при необходимости docs/KNOWLEDGE.md

Gate перехода

- есть карта артефактов;
- есть правила включения контекста;
- есть короткий AGENTS.md;

- есть versioned prompts.

Фаза Execution planning

Цель

Разбить работу на milestone и task slices.

Входы

- все документы предыдущих фаз.

Выходы

- docs/EXEC_PLAN.md
- docs/TASK_BOARD.md
- docs/RISK_REGISTER.md

Gate перехода

- крупные задачи разложены на малые;
- для каждой подзадачи определён done;
- понятны зависимости;
- есть rollback points.

Фаза Controlled implementation loops

Цель

Реализовывать фичи небольшими подтверждаемыми итерациями.

Цикл

1. выбрать задачу из EXEC_PLAN;
2. подключить минимальный контекст;
3. попросить ИИ пересказать задачу;
4. дождаться уточняющих вопросов;
5. согласовать план по файлам;
6. выполнить минимальную реализацию;
7. прогнать build/test/lint/typecheck/schema checks;
8. обновить CHANGE_LOG_DEV и EXEC_PLAN;
9. при необходимости откатиться.

Gate перехода

- текущий slice завершён;
- проверки прошли;
- риски зафиксированы;
- следующий шаг понятен.

Фаза Hardening & evals

Цель

Подтвердить качество и устойчивость результата.

Выходы

- docs/TEST_CATALOG.md
- evals/eval_dataset.jsonl
- docs/QUALITY_SCORECARD.md
- docs/KNOWN_LIMITATIONS.md

Минимум проверок

- unit tests;
- integration tests;
- e2e/browser tests;
- schema validation;
- AI eval cases;
- adversarial cases;
- regression set.

Gate перехода

- критические сценарии проходят;
- нет блокирующих дефектов;
- quality scorecard заполнен;
- ограничения известны и зафиксированы.

Фаза Release & operate

Цель

Подготовить откатываемый и наблюдаемый релиз.

Выходы

- CI/CD pipelines
- docs/RELEASE_CHECKLIST.md
- docs/RUNBOOK.md
- docs/OBSERVABILITY.md
- dashboards / alerts

Gate перехода

- есть staged rollout;
- есть rollback plan;
- включены structured logs, traces и metrics;
- выставлены budget/usage limits;
- опасные действия защищены approvals.

Definition of Ready

- [ ] PRODUCT_BRIEF актуален
- [ ] USER_FLOWS актуален
- [ ] ARCHITECTURE и contracts существуют
- [ ] AGENTS.md создан
- [ ] INDEX.md создан
- [ ] EXEC_PLAN актуален
- [ ] TEST_CATALOG содержит минимум базовых сценариев
- [ ] mock/fixture данные готовы
- [ ] security ограничения понятны
- [ ] build/test/lint/typecheck команды определены

Definition of Done

- [ ] задача выполнена в рамках согласованного task slice
- [ ] build проходит
- [ ] tests проходят
- [ ] contracts/schema валидны
- [ ] риски и ограничения зафиксированы
- [ ] EXEC_PLAN обновлён
- [ ] CHANGE_LOG_DEV обновлён
- [ ] manual checks выполнены
- [ ] при необходимости обновлены docs и decisions

Правила работы с ИИ

- Не просить “сделай всё приложение целиком”.
- Не подавать весь репозиторий в каждый запрос.
- Сначала требовать пересказ задачи.
- Затем требовать уточняющие вопросы.
- Затем требовать план по файлам.
- Только после этого — реализацию.
- После реализации обязательно запрашивать:
- self-review;
- список изменённых файлов;
- результаты тестов;
- оставшиеся риски;
- ручные проверки.
- При неудаче не “лечить чатами”, а откатываться к стабильной точке.

Правила снижения багов

- Всё критичное формализовать схемами.
- Для каждой фичи иметь happy/edge/failure/regression case.
- Не смешивать рефакторинг и новую логику без необходимости.
- Любое изменение контракта требует обновления тестов.
- Любое изменение архитектуры требует записи в DECISIONS.md.

Правила снижения расхода токенов

- Использовать короткий стабильный системный контекст.
- Держать устойчивые инструкции в AGENTS.md / knowledge.
- Подключать только нужные файлы по INDEX.md.
- Разбивать задачи на small task slices.
- Маршрутизировать модели по сложности задач.
- Использовать caching, если префикс стабилен.
- Для длинных сессий применять compaction.
- Для офлайн-задач использовать batch/flex режимы.
- Не исправлять хаос токенами — откатываться.

Минимальный набор метрик

- task success rate
- regression pass rate
- schema validity rate
- tool failure rate
- retry rate
- p95 latency
- cost per successful task
- user-visible defect rate
- rollback frequency
- cache hit rate

Release checklist

- [ ] CI/CD зелёный
- [ ] миграции проверены
- [ ] rollback-план существует
- [ ] structured logs включены
- [ ] traces включены
- [ ] метрики определены
- [ ] критические user flows проверены вручную
- [ ] secret scan и dependency scan выполнены
- [ ] guardrails и approvals настроены
- [ ] budget / usage limits выставлены

Anti-patterns

- “Сначала быстро нагенерим, потом разберёмся”
- “Один огромный prompt на всё”
- “Тесты потом”
- “Если приложение работает у меня — можно релизить”
- “Лучше ещё пять промптов, чем rollback”
- “Можно не вести change log и decisions”
- “Пока оставим секреты как есть”

```
## Краткий чек-лист для печати
```

Ниже — компактная версия, рассчитанная на распечатку и быстрый контроль перед стартом и релизом. Она сознательно короткая: хороший чек-лист должен помогать принимать решение, а не превращаться в ещё один длинный документ. Его содержание соответствует приведённой выше методике и опирается на практики plan-first, contracts-first, eval-driven development, guardrails и observability. [45]

```text
ЧЕК-ЛИСТ «РАЗРАБОТКА ПО С НУЛЯ ЧЕРЕЗ ВАЙБКОДИНГ»

ПЕРЕД НАЧАЛОМ
[ ] PRODUCT_BRIEF заполнен
[ ] USER_FLOWS описаны
[ ] ARCHITECTURE и контракты существуют
[ ] AGENTS.md создан
[ ] INDEX.md создан
[ ] EXEC_PLAN актуален
[ ] TEST_CATALOG создан
[ ] SECURITY_BASELINE создан
[ ] build/test/lint/typecheck команды известны
[ ] есть mock/fixture данные

НА КАЖДУЮ ЗАДАЧУ
[ ] задача разбита на маленький slice
[ ] в контекст включены только нужные документы
[ ] ИИ пересказал задачу
[ ] ИИ задал уточняющие вопросы
[ ] план по файлам согласован
[ ] реализация минимальна
[ ] self-review выполнен
[ ] build/test/schema checks пройдены
[ ] CHANGE_LOG_DEV обновлён
[ ] EXEC_PLAN обновлён

ПЕРЕД РЕЛИЗОМ
[ ] regression suite зелёная
[ ] CI/CD зелёный
[ ] rollback-план есть
[ ] structured logs включены
[ ] traces включены
[ ] основные метрики определены
[ ] guardrails/approvals настроены
[ ] secret scan и dependency scan выполнены
[ ] budget/usage limits выставлены
[ ] известные ограничения зафиксированы
```

Приоритетные источники

Ниже — короткий список источников, на которые опирается этот гайд. Сначала идут первоисточники и официальные документы, затем академические и прикладные исследования, затем крупные кейсы и русскоязычные материалы.

Официальные документы платформ и инженерные гайды: OpenAI Structured Outputs, Function Calling, Evaluation Best Practices, Prompt Caching, Compaction, Tool Search, Batch, Flex Processing, Guardrails/Human Review и Integrations & Observability. [46]

Anthropic: *Effective context engineering for AI agents*, Claude prompt engineering, develop tests, structured outputs, prompt caching, programmatic tool calling, latency guidance. [47]

Google: Vertex AI context caching, function calling и prompt design strategies; Google Cloud guidance по deploy/operate generative AI applications и reliability for AI workloads. [48]

Крупные практические кейсы и IDE/agent guides: Karpathy MenuGen, Replit vibe coding docs и security checklist, Lovable prompting/knowledge/plan mode/testing, Cursor agent best practices и long-running agents, Codex AGENTS.md, workflows, skills, review и best practices. [49]

Академические и прикладные исследования по качеству и безопасности AI-кода: SWE-bench, *Bugs in Large Language Models Generated Code*, *Asleep at the Keyboard?*, *You still have to study*, *Security and Quality in LLM-Generated Code*, METR study on experienced open-source developers. [50]

Русскоязычные материалы как дополнительный контекст, а не замена первоисточникам: Хабр-материалы о вайбкодинге, промптинге и LLM-безопасности. [51]

---

[1] [7] [10] [49] https://karpathy.bearblog.dev/vibe-coding-menugen/
https://karpathy.bearblog.dev/vibe-coding-menugen/

[2] [12] [14] [44] [45] https://docs.lovable.dev/prompting/prompting-one
https://docs.lovable.dev/prompting/prompting-one

[3] [18] [47] https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents
https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents

[4] [23] https://metr.org/blog/2025-07-10-early-2025-ai-experienced-os-dev-study/
https://metr.org/blog/2025-07-10-early-2025-ai-experienced-os-dev-study/

[5] https://docs.lovable.dev/introduction/welcome
https://docs.lovable.dev/introduction/welcome

[6] [20] https://docs.lovable.dev/features/plan-mode
https://docs.lovable.dev/features/plan-mode

[8] https://simonwillison.net/2025/Mar/19/vibe-coding/
https://simonwillison.net/2025/Mar/19/vibe-coding/

[9] https://docs.replit.com/tutorials/how-to-vibe-code
https://docs.replit.com/tutorials/how-to-vibe-code

[11] [51] https://habr.com/ru/articles/942700/
https://habr.com/ru/articles/942700/

[13] https://learn.microsoft.com/en-us/azure/well-architected/ai/mlops-genaiops
https://learn.microsoft.com/en-us/azure/well-architected/ai/mlops-genaiops

[15] [25] [43] https://arxiv.org/abs/2403.08937
https://arxiv.org/abs/2403.08937

16 https://cursor.com/blog/agent-best-practices
https://cursor.com/blog/agent-best-practices

17 46 https://developers.openai.com/api/docs/guides/structured-outputs
https://developers.openai.com/api/docs/guides/structured-outputs

19 32 https://developers.openai.com/api/docs/guides/prompt-caching
https://developers.openai.com/api/docs/guides/prompt-caching

21 https://cursor.com/blog/scaling-agents
https://cursor.com/blog/scaling-agents

22 https://developers.openai.com/codex/learn/best-practices
https://developers.openai.com/codex/learn/best-practices

24 31 33 37 https://developers.openai.com/api/docs/guides/evaluation-best-practices
https://developers.openai.com/api/docs/guides/evaluation-best-practices

26 34 https://docs.github.com/en/actions/get-started/understand-github-actions
https://docs.github.com/en/actions/get-started/understand-github-actions

27 42 https://owasp.org/www-project-top-10-for-large-language-model-applications/assets/PDF/OWASP-Top-10-for-LLMs-v2025.pdf
https://owasp.org/www-project-top-10-for-large-language-model-applications/assets/PDF/OWASP-Top-10-for-LLMs-v2025.pdf

28 https://docs.lovable.dev/features/knowledge
https://docs.lovable.dev/features/knowledge

29 https://developers.openai.com/codex/guides/agents-md
https://developers.openai.com/codex/guides/agents-md

30 https://developers.openai.com/codex/workflows
https://developers.openai.com/codex/workflows

35 https://opentelemetry.io/
https://opentelemetry.io/

36 https://opentelemetry.io/docs/
https://opentelemetry.io/docs/

38 https://docs.replit.com/core-concepts/agent/checkpoints-and-rollbacks
https://docs.replit.com/core-concepts/agent/checkpoints-and-rollbacks

39 40 https://developers.openai.com/api/docs/guides/agents/guardrails-approvals
https://developers.openai.com/api/docs/guides/agents/guardrails-approvals

41 https://developers.openai.com/api/docs/guides/agents/sandboxes
https://developers.openai.com/api/docs/guides/agents/sandboxes

48 https://docs.cloud.google.com/vertex-ai/generative-ai/docs/context-cache/context-cache-overview
https://docs.cloud.google.com/vertex-ai/generative-ai/docs/context-cache/context-cache-overview

50 https://arxiv.org/abs/2310.06770
https://arxiv.org/abs/2310.06770